

Kaitlyn Bedgood
EECS 581
Extracting IPv4 Addresses from Noisy Text AI Disclosure

General Disclosure

I used ChatGPT (GPT-5.6 Sol) OpenAI on September 27, 2026. It was used to develop the program for extracting IPv4 from noisy texts, create test cases, and debug issues I had with the program.

Code Attribution

I first gave ChatGPT a general idea of the program and separate prompt after to give the general requirements.

Prompt:

I need to write a C++ program that reads in a line of text and extracts a single valid IPv4 address.

Prompt:

Requirements:

IPv4 address needs 4 octets separated by periods.

Each octet 1-3 digits and has to be between 0-255

Octet can't have a leading 0 unless it is just 0

Address can have a port after the 4th octet but it will be like :port

Port can be 1-5 digits and the value is 0-65535 and cannot have a leading 0

If there is a colon after the IP address the port has to be valid, if not address and port is rejected

The IP address can be located anywhere in a line of text

Prompt:

Also, only digits, periods, and colons can be part of the valid token.

Prompt:

On success, print: `Extracted IPv4 address: A.B.C.D (decimal value: N, port: P)` where N is the 32-bit decimal value and P is the port number or the literal text none.

Result of Prompts:

These prompts gave me an idea of the initial structure for `extractIPv4` function in `main.cpp`. ChatGPT suggested going through the string character by character. The code was not built at this point, these prompts were just to state the requirements and get a rough idea on how to approach coding the program. After using ChatGPT to help build the `extractIPv4` function, I noticed it was creating code without a way to test it. I added code in `extractIPv4` for temporary testing purposes and a main to run the program:

```
cout << "Value: " << value << endl;
cout << "Digit count: " << digitCount << endl;
```

```
int main()
{
    string test = "192.168.1.1";

    unsigned long address;
    int port;

    extractIPv4(test, address, port);

    return 0;
}
```

After adding that code, I noticed there was an issue with when I tested “2555”, instead of outputting it was invalid it moved down the string to find an octet.

Valid octet: 55

Valid octet: 5

Prompt:

I tested the code with "2555" and it did not reject it. It

did: kaitlyn@LittleLinny:~/git/eecs_581_ivp4\$./main Valid octet: 55 Valid octet: 5

Result of Prompt:

ChatGPT suggested checking the character before the starting position. If the character is a digit, colon, or period the program cannot start a new candidate there.

After working on the other octets, I noticed ChatGPT wanted to keep repeating the same code.

Prompt:

Actually we still need the 3rd octet. The code is very similar though, can we do a loop instead of rewriting the code?

Result of Prompt:

Instead of repeating the same code for four octets, a for loop was used once for each octet. Inside the loop it builds the value of the current octet, counts the digits, checks the value and for leading zeros. After the octet passes the validation check, the value is added to the octets array. After testing the code, I found there was a bug with “1.1.1.1.”, it was passed as valid. The code below was written to correct the bug.

```
if (j < str.length() && str[j] == '.')
{continue;}
```

Prompt:

It works correctly. Let's work on the port now

Result of Prompt:

ChatGPT helped add the possible addition of the port after the fourth octet. The code checked that the port contained 1-5 digits and had a value between 0 and 65535.

Prompt: (I gave the C++ requirements directly in a prompt)

```
C++
// Returns true if a valid address was found, false otherwise.
// On success: outAddress holds the 32-bit value,
// and outPort holds the port number, or -1 if no port was present.
// On failure: outAddress is set to 0 and outPort is set to -1.
bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort);
```

Result of Prompt:

I already gave the requirements for the outAddress needing to hold a 32 bit value of the IPv4 address. ChatGPT helped me build the value from the four validated octets using bit shifting.

Verification Statement

I reviewed the code while working on the program and made sure to understand what each part is doing. I did not have ChatGPT write the entire program at once, instead I worked through it in smaller pieces and tested as I went. Testing as I went helped catch bugs like "2555" and "1.1.1.1." right away and made the rest of the process smoother. I also caught the duplication of code in the extractIPv4 function and created a for loop instead. I understand that the program goes through a string character by character and checks for octets, an optional port, and builds an outAddress. I compiled and tested the program with valid and invalid addresses and ports to make sure the program ran correctly.